Assignment

University Management System

Submited
By Suprem
Lamichhane

University Management System

1. import java.util.Scanner;

```
class Student {
    private String name;
    private int rollno;
    private String faculty;

    public Student (String name, int rollno, String faculty) {
        this.name = name;
        this.rollno = rollno;
        this.faculty = faculty;
    }
    public String getName () {
        return name;
    }
    public int getRollno () {
        return rollno;
    }
    public String getFaculty() {
        return faculty;
    }
    public void setName (String name) {
        this.name = name;
    }
    public void setRollno(int rollno) {
        this.rollno = rollno;
    }
    public void setFaculty ( String faculty) {
        this.faculty = faculty;
    }
```

```
public void displayProfile(){
System.out.println("\n Student profile ");
System.out.println(" Name :        "+name);
System.out.println(" Rollno :      "+rollno);
System.out.println(" faculty :     "+faculty);
}
public static void main(String[] args){
    Scanner sc = new Scanner(System.in)
     System.out.println("Enter name:");
      String name = sc.nextLine();
     System.out.println("Enter rollno:");
       int rollno = sc.nextInt();
         sc.nextLine();
     System.out.println("Enter faculty");
      String faculty = sc.nextLine();
       student s = new Student(name, rollno, faculty);
       s.displayprofile();
         sc.close();
      }
    }
```

```
2. public class Employee-info {
     static class Employee {
       protected String name;
       protected int employeeid;
       protected double salary;

       public Employee (String name, int employeeid, double salary)
       {
         this.name = name;
         this.employeeid = employee.id;
         this. salary = salary;
       }
       public void display details() {
          System.out.println(" Employee information ");
          System.out.println(" Employee name :  "+ name);
          System.out.println(" Employee id :  " + employeeid);
          System.out.println(" Employee salary : Rs. " + salary);
       }
    }
    static class Teacher extends Employee {
       private String subject;
       private String destination;
       public Teacher (String name, int employeeid, double
              salary, String subject, String designation) {
        super (name, employeeid, Salary);
        this.subject = subject;
        this. designation = designation;
    }
    @Override
    public void displaydetails () {
     System.out.println("   teaching staff ");
       super. display details ();
```

```
System.out.println(" Subject :   "+subject);
System.out.println(" Designation :   " + designation);
}
}
static class Administration extends Employee {
    private String department;
    private String role;
    public Administration (String name, int employeeid, double
    salary, String department, String role) {
      super(name, employeeid, salary);
        this.department = department;
        this.role = role;
    }
    @Override
    public void displaydetails() {
     System.out.println(" Administration staff ");
       super.displaydetails();
    System.out.println("Department :   " + department);
    System.out.println(" Role :   " +role);
    }
  }
  public static void main (String[] args) {
  Teacher t1 = new Teacher(" Sanju Shrestha", 2565128, 5000
     "object-oriented programming", "Lecturer");
  Administration a1 = new Administration(" Aahana
  pokhrel", 1010.65, 30000, "Finance", "Head Accountant");
      t1.displaydetails();
       System.out.println();
        a1.displaydetails();
  }
}
```

3)
```
import java.util.Arraylist;
public class course_register {
    static class Course {
        private String courseName;
        private String Course Code;

        public Course (String CourseName, String CourseCode) {
        this.CourseName = CourseName;
        this.Course Code = CourseCode;
        }
    }
    static class Students {
        private String name;
        Arraylist <Course> courses = new ArrayList<>();
        public Students (String name) {
          this.name = name;
        }
        ~~public Students (String~~
        void registerCourses (Course c) {
        courses.add(c);
        }
        void displayCourses () {
         System.out.println("courses of " +name);
           for (Courses c : Courses)
               System.out.println("[" +c.CourseCode+ "] ~~+ Course~~
               +c CourseName);
        }
    }
    public static void main (String[] args) {
     Students s = new Students("Sushan gyawali");
s.registerCourse (new Course ("Object-oriented programming", BIS" T6035"))
s.registerCourse (new Course ("Operating System", OPS-25"));
```

```
s.registerCourse(new Course("Computer Organization", "ECM-381"));
s.displayCourses();
}
}
```

4)
```
abstract class Student1 {
    abstract void calculateFee();
}
class Undergraduate extends Student1 {
  @Override
    void calculateFee() {
        System.out.println("Undergraduate fee = Rs. 60,000");
    }
}
class Graduate extends Student1 {
  @ Override
    void calculateFee() {
        System.out.println ("Graduate fee = Rs. 90,000");
    }
}
public class Main {
    public static void main(String[] args) {
        Student1  s1 = new Undergraduate();
        Student2  s2 = new Graduate();
        s1.calculatefee ();
        s2.calculate fee ();
    }
}
```

1). 5)

```
interface ResultProcessor {
    String calculateGrade(double marks);
}
class EngineeringDepartment implements ResultProcessor {
  @Override
  public String calculateGrade(double marks) {
    if(marks >90)
        return "A+";
  else if (marks >=80)
        return " A";
  else if  (marks >=70)
         return "B+";
  else if (marks >=60)
          return "B";
  else
      return "F";
    }
  }
class ManagementDepartments implements ResultProcessor {
  @Override
   public String calculateGrade (double marks) {
    if (marks>=85)
      return "Distinction";
   elseif (marks>=70)
        return "First Division";
   elseif (marks>=55)
         return "Second Division";
   else
         return "Fail";
  }
}
```

```
public class GradeMain {
    public static void main(String[] args) {
        ResultProcessor eng = new EngineeringDepartment();
        ResultProcessor mgmt = new ManagementDepartment();
        System.out.println("Engineering Grade (85): " +
                eng.calculateGrade (85));
        System.out.println("Management Grade (85): " +
        mgmt.calculateGrade (85));
    }
}
```

b)
```
class Member {
    String membername;
    Member (String memberName) {
    this.memberName = memberName;
    }
    int borrowinglimit() {
     return 2;
     }
    void displayInfo() {
     System.out.println("Member : " + membername);
     System.out.println(" Borrow limit: " + borrowinglimit()
            + " books");
  }
}
class StudentMember extends Member {
    StudentMember(String name) {
        super(name);
     }
  @Override
    public int borrowinglimit() {
```

```
return 3;
  }
}
class TeacherMember extends Member {
      TeacherMember (String name) {
      super (name);
      }
   @Override
   public int borrowinglimit() {
   return 10;
   }
 }
public class LibraryMain {
      public static void main(String[] args) {
      Member sm = new StudentMember("Aahana");
      Member tm = new TeacherMember ("Sarju Shrestha");
    sm.displayInfo();
     System.out.println();
     tm.displayInfo();
   }
}
```

7) class Attendence {

```
class Attendence {
  int attenced, total;
  Attendence (int attended, int total) {
   this.attended = attended;
   this.total = total;
    }
   double calculateAttendence() {
    return (attended * 100) / total;
   }
   void display() {
   System.out.printf("Attendence: %.2f%%\n", calculateAttendence
     ());
   }
 }
 class EngineeringAttendence extends Attendence {
     EngineeringAttendence (int a, int t) {
      super(a, t);
      }
      @Override
      double calculateAttendence () {
        System.out.print("[Engineering - Min 75%]");
        return super.calculateAttendence();
      }
    }
    class MedicalAttendence extends Attendence {
     MedicalAttendence (int a, int t) {
      super (a, t);
      }
     @Override
     double calculateAttendence () {
     System.out.print("[Medical - Min 80%]");
      return super.calculateAttendence();
```

```
    }
    }
public class AttendenceMain {
    public static void main(String[] args){
  Attendence eng = new EngineeringAttendence(35,50);
  Attendence med = new MedicalAttendence (44,50);
  eng.display();
  med.display();
  }
}
```

ndence

8)
```
interface Notification {
   void sendNotification (String recipient, String message);
}
class EmailNotification implements Notification {
  @Override
  public void sendNotification (String recipient, String message){
   System.out.println("[Email]");
   System.out.println("To : " + recipient);
   System.out.println("Message: " + message);
  }
}
class SMSNotification implements Notification {
  @Override
  public void sendNotification (String recipient, String
    message) {
   System.out.println("[SMS]");
   System.out.println("To : " + recipient);
   System.out.println(" Message: " + message);
  }
}
```

```
public class NotificationMain {
 public static void main [String [] args] {
  Notification email = new EmailNotification ();
  Notification sms = new SMSNotification ();
 email.sendNotification ("anjila@kfa.edu.np", "Your
  assignment is due Monday.");
  System.out.println();
  sms.sendNotification ("+977-9801010101", "Assignment
  due Monday!");
 }
}
```

9)
```
class Student2 {
   private static int counter = 1000;
   private int studentID;
   private String name;
   Student2 (String name) {
    this.name = name;
    this.studentID = ++counter;
    }
   void display () {
    System.out.println ("ID: " + studentId "| Name: "
      + name); }
      }
   }
   public class UniqueIdMain {
        public static void main (String[] args)
     {
       Student2 s1 = new Student2 ("Anjila");
       Student2 s2 = new Student2 ("Ram");
       Student2 s3 = new Student3 ("Sita");
        s1.display ();
```

```
        s2. display();
        s3. display();
        }
    }

10) abstract class Scholarship {
        String ~~class~~ StudentName;
        double gpa;
        Scholarship (String n, double g) {
        student name = n;
        gpa = g;
        }
    abstract boolean isEligible();
    abstract double getAmount();
      void displayResult() {
       System.out.println("Student: "+ studentName);
       System.out.println("GPA : %.1f%n", gpa);
       System.out.println(" Eligible : "+(isEligible()?"Yes":
                  "No"));

     if (isEligible())
        System.out.printf("Amount : NPR %.2f%n",
                     getAmount());

    }
  }
  class MeritScholarship extends Scholarship
      {
      MeritScholarship (String n, double g) {
      super (n, g);
      }
    @Override
      public boolean isEligible() {
      return gpa >= 3.7;
      }
```

```
@Override
public double getAmount () {
  return 50000.00;
  }
}
class NeedBasedScholarship extends Scholarship {
    double familyIncome;
    NeedBasedScholarship (String n, double g,
     double f) {
       super (n,g);
       familyIncome = f;
       }
    @Override
    public boolean isEligible () {
    return familyIncome < 300000 && gpa >= 2.5;
     }

   @ Override
     public double getAmount () {
     return 3000.00;
      }
 }
 public class ScholarshipMain {
      public static void main (String[] args) {
       Scholarship m = new MeritScholarship ("Aahana",
           3.85);
      Scholarship n = new NeedBasedScholarship ("Anshu",
            3.1, 250000);
      Scholarship x = new MeritScholarship ("Sushan", 3.2);
      m.displayResult();
      n.displayResult();
      x.displayResult();
     }
 }
```

Output

```
Run   Student ×

/Library/Java/JavaVirtualMachines/jdk-26.jdk/Contents/Home/bin/java
Enter name:
ram chandra sharma
Enter rollno:
099
Enter faculty:
bcsit

   Student profile
Name:  ram chandra sharma
Rollno: 99
faculty: bcsit

Process finished with exit code 0
```

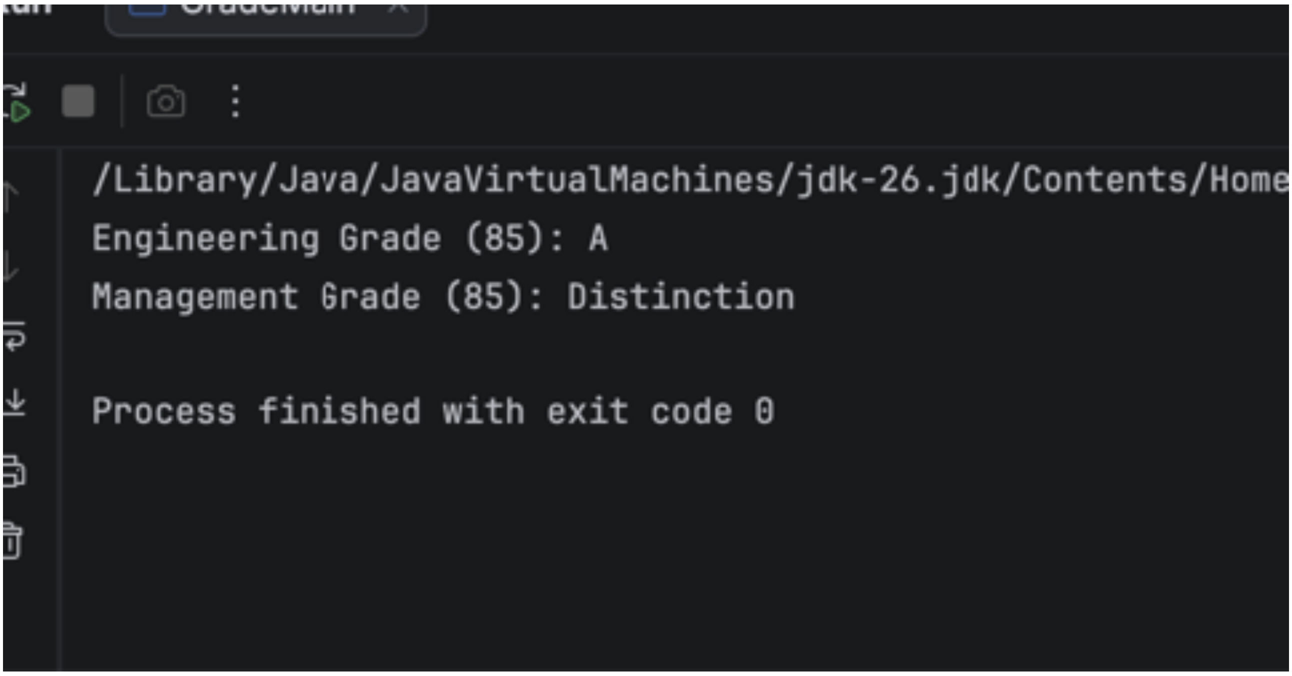
```
      teaching staff
      Employee information
 Employee name: Sanju shreastha
 Employeeid: 25651287
Employee salary: Rs. 50000.0
Subject:  object-oriented programming
Designation: Lecturer

      Administration staff
      Employee information
 Employee name: Aahana pokhrel
 Employeeid: 1010165
Employee salary: Rs. 30000.0
Department:  finance
Role: head accountant

Process finished with exit code 0
```

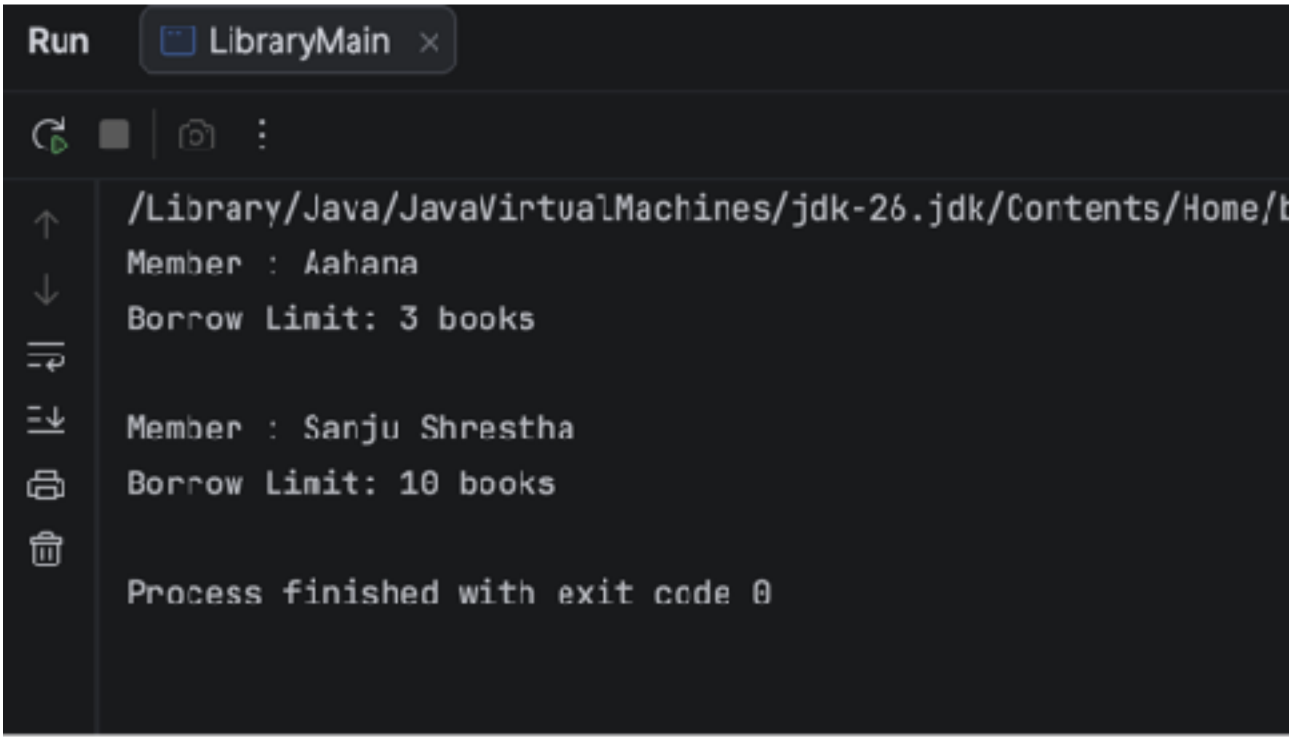
Run course_register

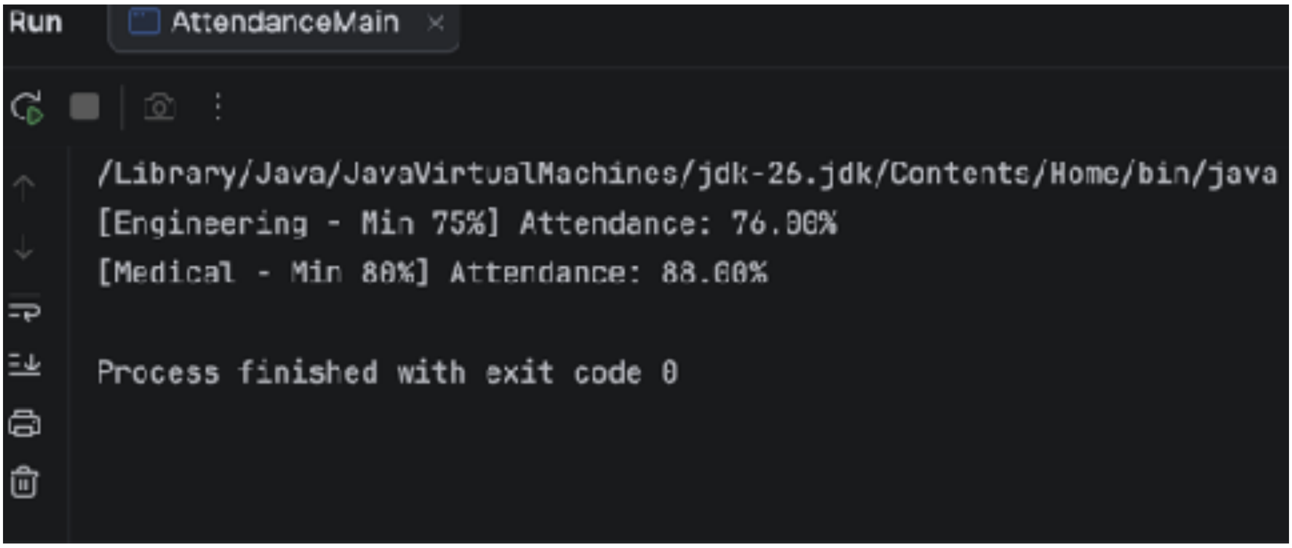
```
/Library/Java/JavaVirtualMachines/jdk-26.jdk/Contents/Home/bin/java
courses of Sushan gyawali
[Bcsnt6033]Object-oriented programming
[OPS-252]Operating System
[ECM-383]Computer Organization

Process finished with exit code 0
```

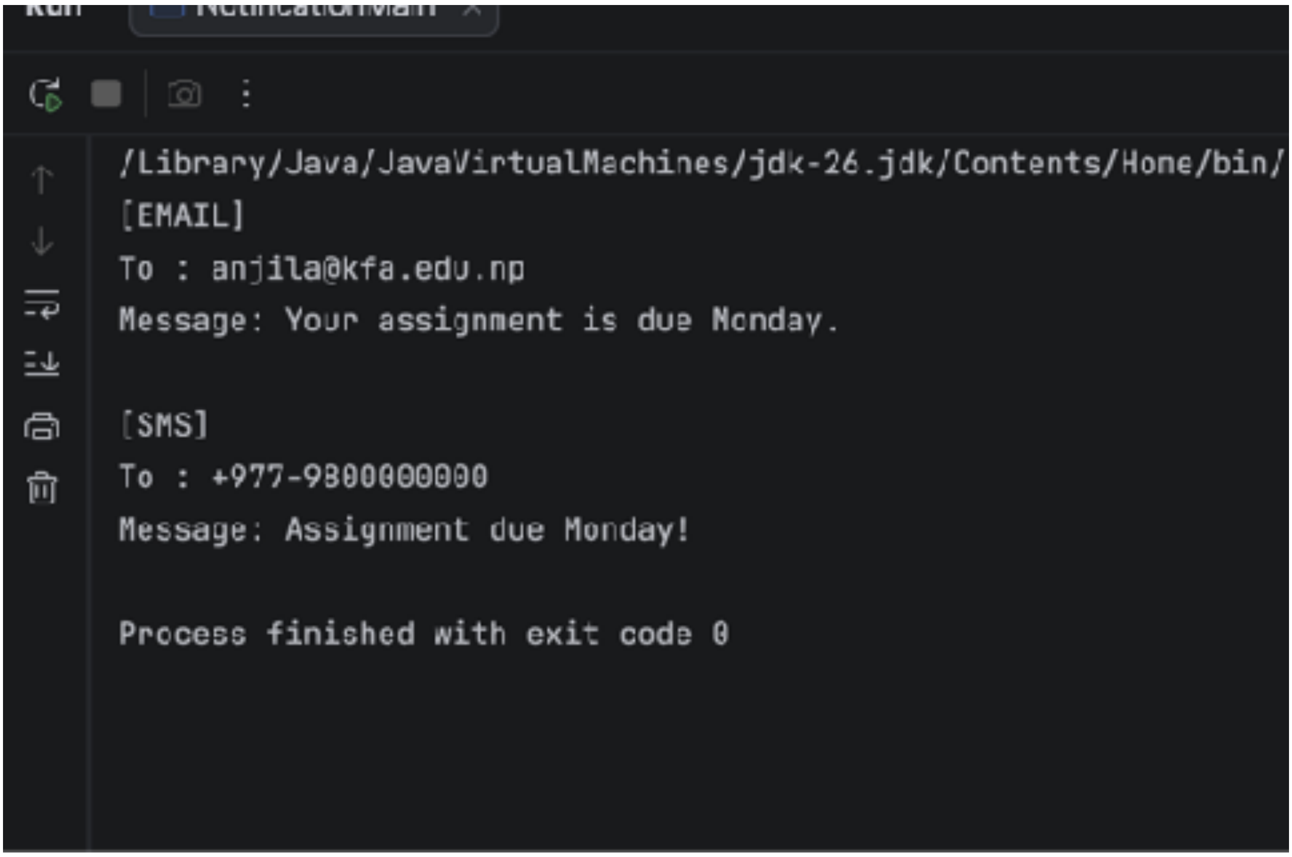
Run Main

```
/Library/Java/JavaVirtualMachines/jdk-26.jdk/Conte
Undergraduate Fee = Rs. 60,000
Graduate Fee = Rs. 90,000

Process finished with exit code 0
```


/Library/Java/JavaVirtualMachines/jdk-26.jdk/Contents/Home
Engineering Grade (85): A
Management Grade (85): Distinction
Process finished with exit code 0


Run
LibraryMain
/Library/Java/JavaVirtualMachines/jdk-26.jdk/Contents/Home/
Member : Aahana
Borrow Limit: 3 books
Member : Sanju Shrestha
Borrow Limit: 10 books
Process finished with exit code 0


Run
AttendanceMain
/Library/Java/JavaVirtualMachines/jdk-26.jdk/Contents/Home/bin/java
[Engineering - Min 75%] Attendance: 76.00%
[Medical - Min 80%] Attendance: 88.60%
Process finished with exit code 0


/Library/Java/JavaVirtualMachines/jdk-26.jdk/Contents/Home/bin/
[EMAIL]
To : anjila@kfa.edu.np
Message: Your assignment is due Monday.
[SMS]
To : +977-9800000000
Message: Assignment due Monday!
Process finished with exit code 0

Run UniqueIdMain

```
/Library/Java/JavaVirtualMachines/jdk-26.jdk/Contents/Home/bin/j
ID: 1001 | Name: Anjila
ID: 1002 | Name: Ram
ID: 1003 | Name: Sita

Process finished with exit code 0
```

Run ScholarshipMain

```
/Library/Java/JavaVirtualMachines/jdk-26.jdk/Contents/Home/bin/
Student : Aahana
GPA : 3.9
Eligible : Yes
Amount : NPR 50000.00
----------------------------
Student : Anshu
GPA : 3.1
Eligible : Yes
Amount : NPR 30000.00
----------------------------
Student : sushan
GPA : 3.2
Eligible : No
----------------------------

Process finished with exit code 0
```